

Inicialmente, allá en el 2022, el primer modelo de uso masivo de LLM (CHATGPT Basado en GPT-3.5) tenía una restricción muy particular que se correspondía a la técnica con la cual se entrenan estos modelos.

Cuando realizabamos una consulta sobre un suceso posterior al 2019 el modelo devolvía un mensaje que indicaba que no tenía datos posteriores a dicha fecha, esto es debido a que el modelo GPT-3.5 había sido entrenado con un set de datos que llegaba hasta esa fecha, y a partir de allí no tenía más información indexada para poder utilizar.

Modelos posteriores, así como otros motores como Gemini vinieron a resolver este inconveniente, esto no lo hicieron con un modelo que se volvía a entrenar todos los días, el modelo era el mismo pero lo que hacía era poder AMPLIAR temporalmente su contexto de datos al realizar un scrapping de internet al momento de la consulta.

Si nosotros realizamos una consulta directa a un LLM, sea local o mediante una API y no tiene configurado el acceso a una tool de búsqueda en línea, nos sucederá lo mismo que con aquella versión de ChatGPT, no tendremos datos actualizados.

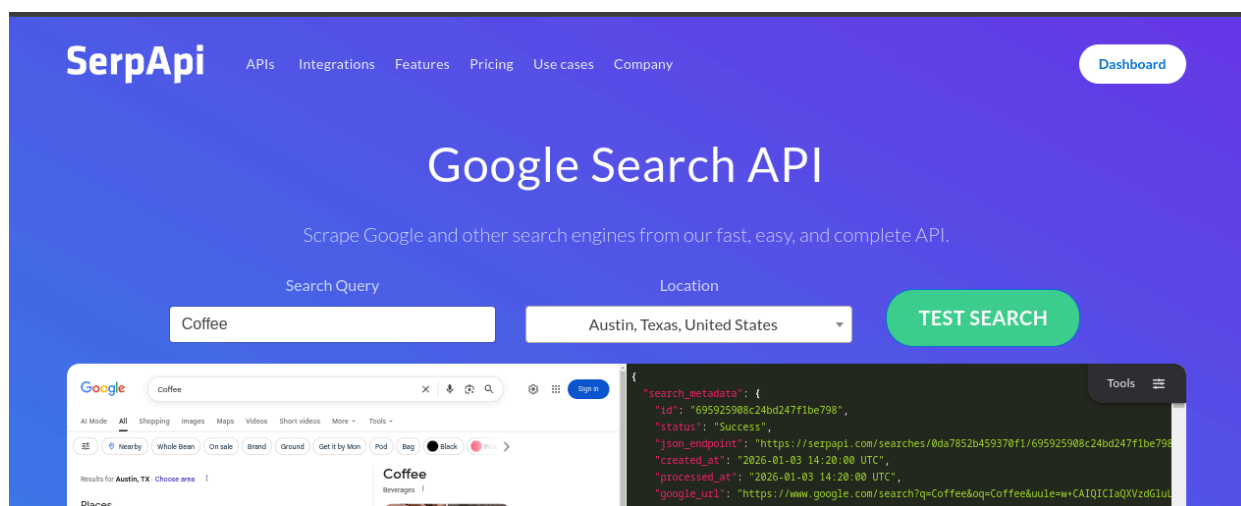
Para poder enmendar esta situación podemos valernos de herramientas que realicen el web scrapping y devuelvan el contexto al modelo.

Para este curso vamos a usar **SerpAPI**, elegimos esta API debido a que al momento de redactar el curso, podíamos utilizarla de forma gratuita para las pruebas y prácticas.

✓ ¿Que es SerpAPI?

SerpApi es una interfaz de programación de aplicaciones (API) que permite a los desarrolladores extraer información de las páginas de resultados de motores de búsqueda (SERP) de manera rápida y estructurada.

<https://serpapi.com/>



Free

\$ 0 / month

Get Started

- ✓ 250 searches per month
- ✓ 50 throughput per hour ?
- ✗ U.S. Legal Shield ?
- ✗ ZeroTrace Mode ?
- ✗ Priority support

Antes de avanzar comento algunas otras alternativas que hay disponibles.

Quiero comentar que una de las principales razones por las cuales comence este curso fue por la llamativa costumbre que tenían otros cursos de SOLO usar modelos pagos, API's que lograban resolver todo milagrosamente y que sin dificultad ni tener que hacer un gran troubleshooting lograbas resolver casi cualquier caso simple.

Creo que es preferible aprender conceptualmente y luego técnicamente el uso de estas herramientas, no como una suite de soluciones/código copy/paste sino, como una tecnología que cambia constantemente. Las API que hoy tenemos disponibles podrían no existir, o tener cambios críticos, en un futuro inmediato y esto nos llevaría a readaptarnos a estos cambios o cambiar completamente de herramienta para solucionar nuestra necesidad.

Comprender de forma clara que necesidad estamos resolviendo con una API, también podemos pensarlo como una tool o incluso extensión, nos permite ganar flexibilidad a la hora de readaptarnos a los cambios, así como también poder migrar a nuevas soluciones a medida que se van desarrollando y la competencia lleva a mejoras técnicas o de precio.

Volviendo a lo que veníamos hablando.

Principales Alternativas a SerpApi (A Junio 26)

ScraperAPI: Excelente para desarrolladores y proyectos a gran escala. Destaca por manejar millones de solicitudes mensuales sin bloquearse, rotar proxies automáticamente y resolver captchas.

Serper (serper.dev): La opción preferida para proyectos de Inteligencia Artificial (LLMs) y agentes. Es muy rápida, ligera y está optimizada principalmente para extraer resultados de texto puro, ubicaciones y respuestas orgánicas. **DataForSEO:** Diseñada específicamente para agencias de marketing y SEO. Ofrece métricas avanzadas (costo por clic, volumen de búsqueda histórico, posición de palabras clave).

Bright Data: Conocida por tener la red de proxies residenciales más grande y estable. Es ideal si requieres geolocalización ultraprecisa y una gran cantidad de puntos de datos extraídos al mismo tiempo.

¿Que es SERP?

Empecemos el día por el desayuno, cuando realizamos una búsqueda en internet con una app de buscador clásico como Google o Bing obtenemos como respuesta (en términos generales) un: SERP son las siglas en inglés de Search Engine Results Page, que se traduce como Página de Resultados del Motor de Búsqueda. Es simplemente la pantalla que devuelve un buscador justo después de que escribiste una consulta o palabra clave.

Normalmente contiene una lista de páginas web, imágenes, videos y otros tipos de contenido que el motor de búsqueda considera relevantes para la consulta del usuario, o (dependiendo del motor y algoritmo usado al momento de la consulta) tiene predefinido mostrar al usuario para el contexto de la consulta.

Los resultados de búsqueda (SERP) son dinámicos y pueden variar en función de muchos factores:

- El motor de búsqueda consultado
- La versión del algoritmo al momento de la consulta
- Las principales búsquedas similares ejecutadas por otros usuarios
- El formato de la búsqueda utilizada

- La geolocalización del usuario
- Historial de búsqueda y preferencias
- Perfil completo del usuario, sea mediante cache o usuario logeado
- Dispositivo (PC, móvil, etc.)

Es decir que es altamente probable que dos usuarios realicen la misma búsqueda y el SERP obtenido sea distinto, incluso un mismo usuario puede repetir la misma búsqueda en distintos momentos y el resultado también tendrá cambios.

Tomando como ejemplo la búsqueda en Google al día de hoy el formato o anatomía de una respuesta de SERP encontramos:

Google search for "que es serp". The search bar shows the query and navigation icons. Below the search bar are tabs for "Modo IA", "Todo", "Imágenes", "Shopping", "Videos", "Noticias", "Videos cortos", "Más", and "Herramientas".

Visión general creada por IA

SERP son las siglas en inglés de *Search Engine Results Page*, que se traduce como **Página de Resultados del Motor de Búsqueda**. Es simplemente la pantalla que devuelve un buscador (como Google, Bing o Yahoo) justo después de que escribes una consulta o palabra clave. [+3](#)

Las SERPs están diseñadas para responder a tu búsqueda de la forma más rápida y útil posible. Por lo general, se componen de los siguientes elementos clave: [WooRank](#)

- **Resultados orgánicos:** Son los enlaces gratuitos que el buscador considera más relevantes para tu búsqueda. Son el objetivo principal de las estrategias de SEO (optimización en motores de búsqueda). [Postedin](#) [Postedin](#) [+3](#)

[Mostrar más](#)

8 sitios

¿Qué son las SERPs y cómo funcionan? - InboundCycle
¿Qué son las SERPs y cómo funcionan? * Qué es una SERP y qué elementos contiene. * Cómo decide...

Qué es SERP y tendencias en buscadores | IEBS Business School
Una SERP, o página de resultados del motor de búsqueda, es la lista de resultados que aparece...

Resultados patrocinados

SerpApi
<https://www.serpapi.com>

SerpApi: Google Search API | Google Search API

SerpApi is a real-time API to scrape Google Autocomplete. Best way to scrape...

[What is a SERP?](#) · [Learn about SERP](#) · [FAQ](#) · [Google Search API](#) · [Features](#) · [Pricing](#)

Más de 1 millón de visitas en el último mes

Hostinger
<https://www.hostinger.com>

Qué es SEO: guía completa para mejorar tu visibilidad en línea

Descubre cómo crear un hipervínculo en WordPress y Microsoft Office. Conoce las ventajas de crear hipervínculos en WordPress y 5 consejos para mejorarlos.

[WordPress Administrado](#) · [Creador de webs con IA](#) · [Hosting WP Administrado](#) · [Transferir tu dominio](#)

TikTok
<https://getstarted.tiktok.com> > [tiktok](#) > [for_business](#)

Create an Ad in Minutes - Register Now & Get Ad Credits

Hacer nueva captura de p



SerpApi

<https://serpapi.com> > blog > what-is-a-serp

¿Qué es una SERP?

19 sept 2024 — Una página de resultados de un motor de búsqueda (SERP, por sus siglas en inglés) es una página web que aparece como respuesta a una consulta en ...

Traducido por Google - [Ver original \(English\)](#)

Más preguntas

¿Qué significa SERP?



¿Cuál es la SERP?



¿Cuál es la diferencia entre SEO y SERP?



¿Qué significa SERP en inglés?



En las imágenes podemos destacar en una primer instancia varias secciones que conforman al día de hoy los resultados de búsqueda, es decir el SERP de Google. Incluso es llamativo ver como los resultados organicos, es decir el listado de URL que mejor match hacen con nuestra pregunta recién aparecen luego de varias secciones.

La estructura actual también tiene cambios según tipo de consulta realizada, puedes aparecer nuevos recuadros de datos geolocalizados con mapas, resultados de APIs como por ejemplo de clima o financieras.

¿Como funcionan por debajo estas API?

Si bien estas API nos devuelven todo resuelto, podemos producir nuestro propio sistema de scrapping, sea para no depender de APIs externas o para poder aprender como funcionan.

Para hacer un sistema básico podemos usar la librería **googlesearch-python**

Vamos a probar un código simple pero funcional:

Primero instalamos la librería `!pip install ddgs`

Esta librería permite un scrapping con duckduckgo (un motor como podría ser Google o Bing), utilice esta librería debido a que las versiones de librerías libres que trabajan con Google está teniendo (al día de hoy) un bloqueo en las consultas y no devuelven resultados.

Luego usamos el siguiente código

```
# Se van a ignorar Warnings asociados a las versiones en uso
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
warnings.filterwarnings("ignore", category=ResourceWarning)
```

```

# El scrapping se ejecuta en 3 etapas
# 1) Realizamos la búsqueda mediante ddgs
# 2) Descargamos los datos en memoria mediante requests
# 3) Formateamos el HTML con BeautifulSoup para extraer los datos específicos
import requests
from bs4 import BeautifulSoup
from ddgs import DDGS

def buscador(query:str)->str:
    """
    Función para realizar una búsqueda en internet y extraer el resultado.
    in:
        query: str
    out:
        url: str
        title: str
        text: str
    """

    # Definimos el header que va a recibir como meta dato el motor de búsqueda
    headers = {"User-Agent": "Mozilla/5.0"}

    # Corremos el método de duckduckgo para la búsqueda en internet
    with DDGS() as ddgs:
        results = list(ddgs.text(query, max_results=1))

    # Extraemos la url de la búsqueda
    r = results[0]
    url = r["href"]

    # Con la URL resultado de la búsqueda, descargamos el html con requests
    response = requests.get(url, headers=headers)
    soup = BeautifulSoup(response.text, "html.parser")

    for tag in soup(["script", "style"]):
        tag.decompose()

    # Extraemos el título y el texto
    title = soup.title.string if soup.title else ""
    text = " ".join(soup.get_text().split())

    return (url, title, text)

```

Se puede probar el código directamente desde Colab o descargando en local

[01 Ejecucion de codigo busqueda en internet](#)

Este código nos puede servir como una herramienta para brindarle al LLM un contexto actualizado sobre el cual trabajar, esto lo logramos al pasarle como parte del prompt la información obtenida con la consulta anterior.

Podemos ampliar este tema ya sea usando IA o con páginas como:

<https://roundproxies.com/blog/how-to-use-googlesearch/>

<https://roundproxies.com/blog/scrape-google-search-results/>

✓ Prueba de LLM con web scrapping

Ahora vamos a unir dos partes de código que ya vimos:

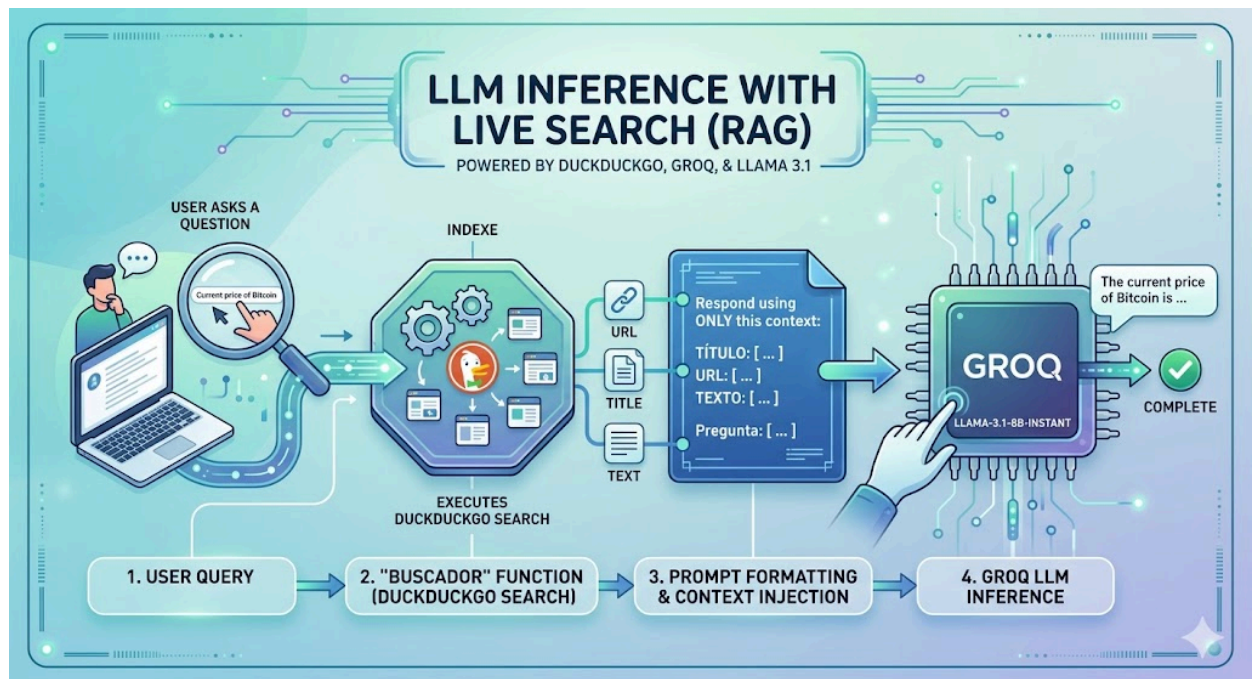
1. Búsqueda en la web de información
2. Consulta a un LLM

El punto de unión entre estas dos partes va a ser el prompt.

De esta forma ampliamos la base de conocimiento del LLM para poder responder

Podemos ver y ejecutar el código desde

[01 Ejemplo codigo LLM con web scrapper propio](#)



✓ Prueba de LLM con SERPAPI

Se puede ver y ejecutar el código completo desde:

[01 Ejemplo_codigo_busqueda_internet_SERPAPI](#)

La arquitectura es muy similar a la anterior, incluso vamos a reciclar la sección de código que realiza la llamada al modelo llama desde Groq. Lo que vamos a cambiar es la etapa del BUSCADOR, en donde ahora vamos a valernos de una api externa que es mas robusta y presenta los resultados de una forma mas eficiente y ordenada.

De esta forma el gran cambio que tenemos es:

```
# El scrapping se ejecuta en 2 etapas
# 1) Pasamos la busqueda a la web https://serpapi.com/search
# 2) Descargamos los datos en memoria mediante requests

import requests

def buscador_con_serpapi(query:str)->str:
    """
    Función para realizar una busqueda en internet y extraer el resultado.
    in:
        query: str
    out:
        url: str
        title: str
        text: str
    """

    #URL de la pagina serpapi adonde vamos a enviar la consulta
    url = "https://serpapi.com/search"

    # Parametros necesarios: cual es la query, apikey que generamos en la p
    params = {
        "q": query,
        "api_key": SERAPI,
        "engine": "google"
    }

    # Descargamos la información generada por la pagina serpapi
    response = requests.get(url, params=params)
    data = response.json()

    # La información devuelta tendra formato JSON y se debera consultar en
    r = data["organic_results"][0]

    # Tomamos los valores de titulo,link y texto/cuerpo para el primer resu
    title = r["title"]
    link = r["link"]
```



```
text = r.get("snippet", "")
```

```
return (url, title, text)
```

Si te quedan preguntas no dudes en contactarme

arielmeragelman@gmail.com

Lic. Sebastian A. Meragelman